SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Internet Programming **COURSE CODE: CSA43**

COURSE OUTCOME COVERED

CO-2: **Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)**

Assessment Tool: Integrated Experiments

Assessment Tool Weightage: 40%

EXPERIMENTS: 3 Total Marks: 30

CO-2 – Integrated Experiments

Name of the Student : SONIAH N

Register Number : 192311012

Code of Conduct:

I **SONIAH N - 192311012** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

All three experiments were coded as plain HTML/CSS/JS files (VS Code) and opened in Google Chrome. The screenshots were taken with the DevTools device toolbar at 375 px (mobile), 768 px (tablet) and 1280 px (desktop).

Experiment 1: Responsive Layout Design

Aim: To redesign a non-responsive retail page ("ShopEasy") so that it lines up properly on mobile, tablet and desktop using CSS Grid, Flexbox and media queries.

(a) Why the existing layout is not responsive

I first opened the old version of the page in Chrome's mobile view. The screenshot below shows what happens: the whole page shrinks to a tiny size, the nav links break into an odd second row, and the product boxes float out of line.



Fig 1.1 – Old layout on a 375 px mobile screen

Looking at the old CSS, these were the main causes:

1. **No viewport meta tag.** Without `<meta name="viewport">`, mobile browsers assume a ~980 px wide page and zoom out, so text becomes unreadable.
2. **Fixed pixel widths.** The header and footer were `width: 1200px` and each card was `350px`. These never shrink, so the page overflows on small screens.
3. **Float-based layout.** The nav and cards used `float: left`. Floats wrap unpredictably when there isn't enough room, which caused the uneven rows.
4. **No media queries.** The same layout was applied at every screen size.
5. **Hard-coded heights and margins.** `height: 200px` and `margin: 20px` on every card gave uneven spacing once things started wrapping.
6. **Default box-sizing.** Padding was added on top of the fixed widths, making elements even wider than intended.

(b) Responsive layout using Grid and Flexbox

I rebuilt the page with a **mobile-first** approach. The outer page uses **CSS Grid with named areas** (header, nav, main, footer), the nav links use **Flexbox**, and the product cards sit in an inner grid. Main code:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
* { box-sizing: border-box; margin: 0; padding: 0; }

.container {
  display: grid;
  grid-template-areas: "header" "nav" "main" "footer";
  min-height: 100vh;
}
header { grid-area: header; background: #2c3e50; color: #fff; padding: 16px 20px; }
nav    { grid-area: nav; background: #34495e; }
```

```
main   { grid-area: main; padding: 20px; }
footer { grid-area: footer; background: #2c3e50; color: #ccc; text-align: center; padding:
14px; }

nav ul { list-style: none; display: flex; flex-wrap: wrap; justify-content: center; }
nav a  { display: block; color: #fff; text-decoration: none; padding: 12px 16px; }

.products { display: grid; grid-template-columns: 1fr; gap: 20px; }
.card {
  background: #fff; border-radius: 8px; padding: 16px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);
  display: flex; flex-direction: column; justify-content: space-between;
}

/* Tablet */
@media (min-width: 600px) {
  .products { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop */
@media (min-width: 1024px) {
  .container {
    grid-template-columns: 200px 1fr;
    grid-template-areas: "header header" "nav main" "footer footer";
  }
  nav ul { flex-direction: column; }
  .products { grid-template-columns: repeat(3, 1fr); }
}
</style>

<div class="container">
  <header><h1>ShopEasy</h1></header>
  <nav><ul><li><a href="#">Home</a></li> ... </ul></nav>
  <main><section class="products"> <div class="card">...</div> ... </section></main>
  <footer>© 2026 ShopEasy</footer>
</div>
```

(c) How the layout adapts (media queries)

Screen	Width used	Breakpoint	What changes
Mobile	375 px	default (no query)	One column of cards, nav links in a horizontal row that wraps
Tablet	768 px	`min-width: 600px`	Cards switch to 2 columns, bigger heading
Desktop	1280 px	`min-width: 1024px`	Nav moves to a 200 px left sidebar, cards in 3 columns

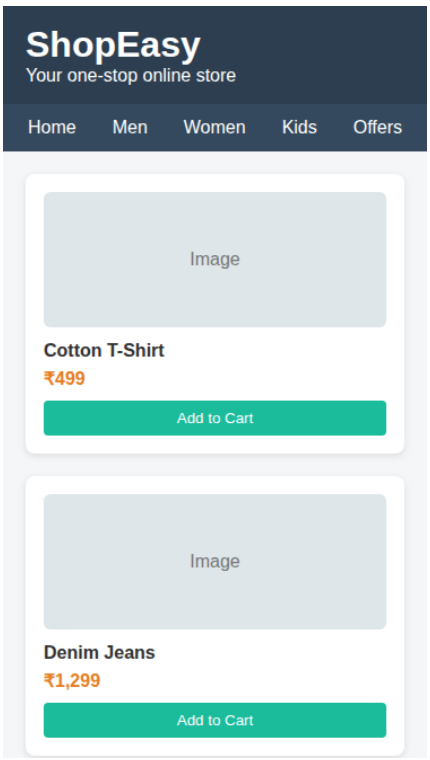


Fig 1.2 – Output on mobile (375 px)

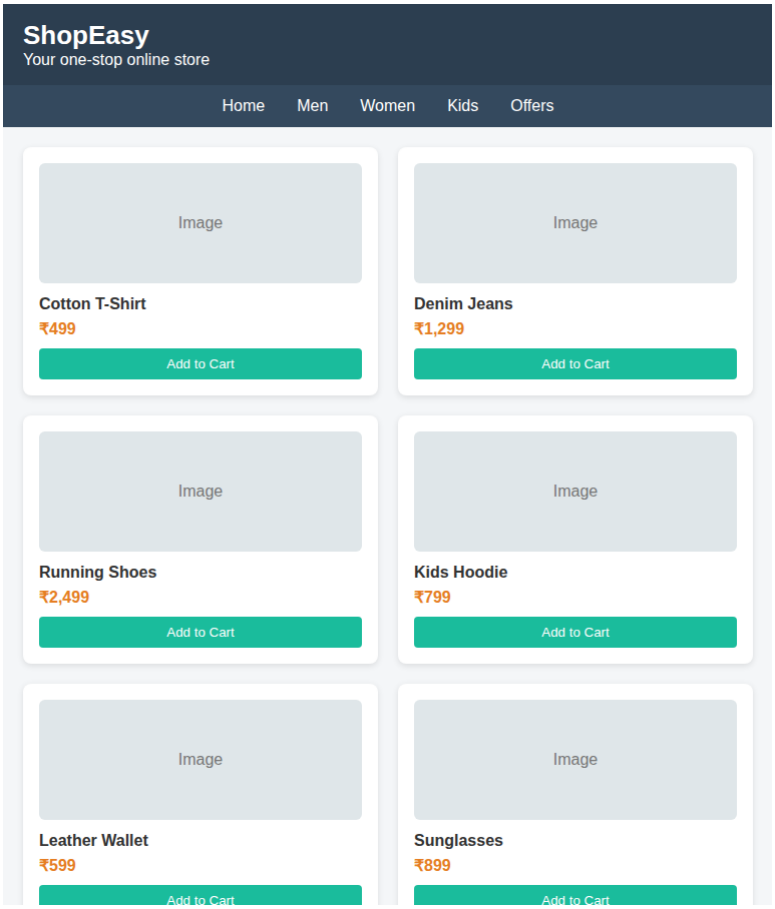


Fig 1.3 – Output on tablet (768 px)

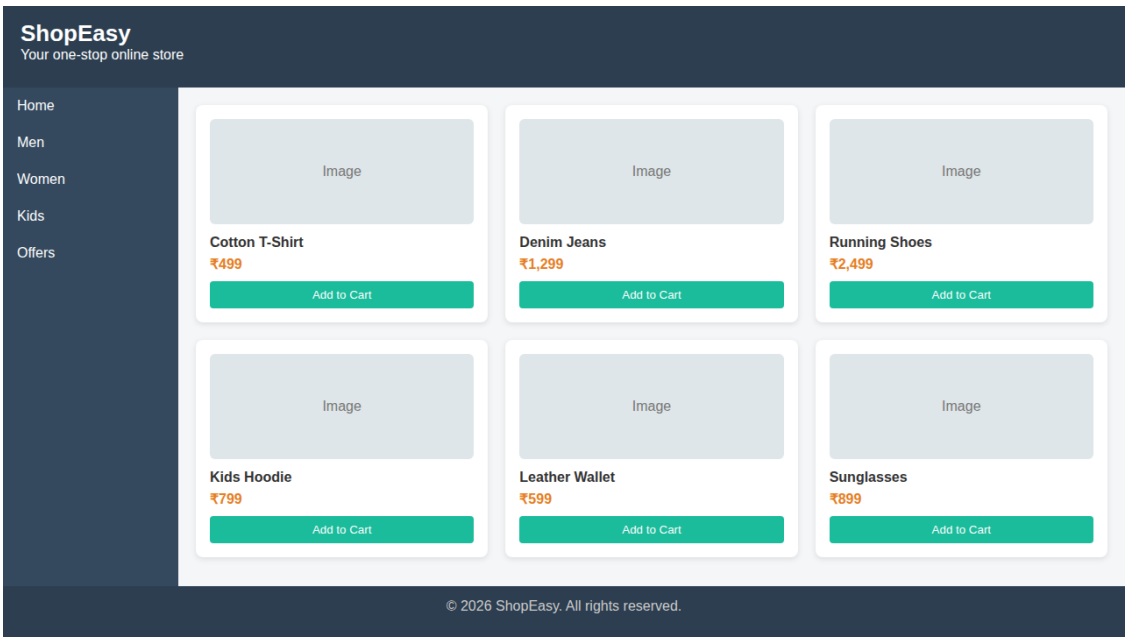


Fig 1.4 – Output on desktop (1280 px)

Only the grid template changes at each breakpoint. The HTML stays exactly the same, which is the main advantage of using grid areas.

(d) Spacing, alignment and distribution

- `gap: 20px` on the product grid gives equal spacing between cards in both directions, so I didn't need margins on individual cards.
- `grid-template-columns: repeat(n, 1fr)` shares the available width equally, so all cards in a row are the same size.
- `justify-content: center` with `flex-wrap: wrap` keeps the nav links centred on mobile and lets them wrap cleanly.
- Inside each card, `flex-direction: column` with `justify-content: space-between` keeps the "Add to Cart" button at the bottom even if the product names have different lengths.
- `box-sizing: border-box` makes the padding part of the width, so cards never spill out of their column.
- I kept padding consistent (16–20 px) across header, main and cards for a uniform look.

(e) Evaluation and further improvements

I tested the page at 375, 768 and 1280 px and also by dragging the browser window. There was no horizontal scrolling at any size, the text stayed readable, and the cards always lined up in neat rows. The mobile-first approach also kept the CSS short, since the base styles already work for phones.

Things that could still be improved:

- Use `repeat(auto-fit, minmax(220px, 1fr))` so the number of columns adjusts automatically without extra breakpoints.
- Replace the nav row with a hamburger menu on very small phones to save vertical space.
- Use real product images with `srcset` / `loading="lazy"` so mobiles download smaller images.
- Use `rem` and `clamp()` for font sizes so text scales smoothly instead of jumping at breakpoints.
- Run a Lighthouse audit to check accessibility and colour contrast.

Experiment 2: Form Validation using JavaScript

Aim: To build a registration form that rejects invalid email IDs and phone numbers using HTML attributes, Regular Expressions and real-time error messages.

(a) Issues in the existing validation

The original form was accepting entries like `abc@` and `12345` without any complaint. On checking the code I found:

1. **Wrong input types.** Both fields were plain `type="text"`, so the browser did no checking of its own.
2. **No validation at all on the client side.** The form submitted whatever was typed.
3. **Weak or missing patterns.** Even a basic check like "contains @" lets through `a@b`, and nothing stopped letters or wrong-length numbers in the phone field.
4. **No feedback to the user.** The user had no idea anything was wrong until bad data was already stored.
5. **No `required`, `maxlength` or `pattern` attributes**, so empty and over-long values were also allowed.
6. **No server-side check.** Client-side validation can be bypassed, so relying only on the browser is not safe.

(b) Input fields with proper HTML attributes

Attribute	Used on	Why
`type="email"`	Email	Built-in email format check and the @ keyboard on mobiles
`type="tel"` + `inputmode="numeric"`	Phone	Opens the number pad on phones
`required`	Both	Field cannot be left empty
`maxlength="10"`	Phone	Can't type more than 10 digits
`pattern="[6-9][0-9]{9}"`	Phone	HTML-level backup of the regex
`placeholder`	Both	Shows the expected format
`autocomplete="email"`	Email	Lets the browser autofill

I added `novalidate` on the form so the browser's own popups don't appear and my custom messages are shown instead.

(c) JavaScript validation with Regular Expressions

- **Email:** `/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/` – a username part, then `@`, then a domain, then a dot and at least 2 letters (`.com`, `.in`, `.edu`).
- **Phone:** `/^[6-9]\d{9}$/` – exactly 10 digits starting with 6, 7, 8 or 9 (the format for Indian mobile numbers).

(d) Real-time error messages

The check runs on every `input` event, so the message updates as the user types. The border turns red or green, and the Register button stays disabled until both fields are valid. Main code:

```
<form class="form-box" id="regForm" novalidate>
  <label for="email">Email</label>
  <input type="email" id="email" placeholder="name@example.com" required
autocomplete="email">
  <span class="error" id="emailError"></span>

  <label for="phone">Phone Number</label>
  <input type="tel" id="phone" placeholder="10-digit mobile number" maxlength="10"
         pattern="[6-9][0-9]{9}" inputmode="numeric" required>
```

```
  <span class="error" id="phoneError"></span>

  <button type="submit" id="submitBtn" disabled>Register</button>
  <p id="msg"></p>
</form>

<script>
const email = document.getElementById("email");
const phone = document.getElementById("phone");
const emailError = document.getElementById("emailError");
const phoneError = document.getElementById("phoneError");
const submitBtn = document.getElementById("submitBtn");

const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
const phoneRegex = /^[6-9]\d{9}$/;

function checkEmail() {
  if (email.value === "") {
    emailError.textContent = "Email is required";
  } else if (!emailRegex.test(email.value)) {
    emailError.textContent = "Enter a valid email (e.g. name@gmail.com)";
  } else {
    emailError.textContent = "";
  }
  setState(email, emailError.textContent === "");
}

function checkPhone() {
  phone.value = phone.value.replace(/\D/g, "");   // digits only
  if (phone.value === "") {
    phoneError.textContent = "Phone number is required";
  } else if (!phoneRegex.test(phone.value)) {
    phoneError.textContent = "Must be 10 digits and start with 6, 7, 8 or 9";
  } else {
    phoneError.textContent = "";
  }
  setState(phone, phoneError.textContent === "");
}

function setState(input, ok) {
  input.classList.toggle("valid", ok);
  input.classList.toggle("invalid", !ok);
  submitBtn.disabled = !(emailRegex.test(email.value) && phoneRegex.test(phone.value));
}

email.addEventListener("input", checkEmail);
phone.addEventListener("input", checkPhone);

document.getElementById("regForm").addEventListener("submit", function (e) {
  e.preventDefault();
  checkEmail(); checkPhone();
  if (!submitBtn.disabled) {
    document.getElementById("msg").textContent = "Registration successful!";
  }
});
</script>
```

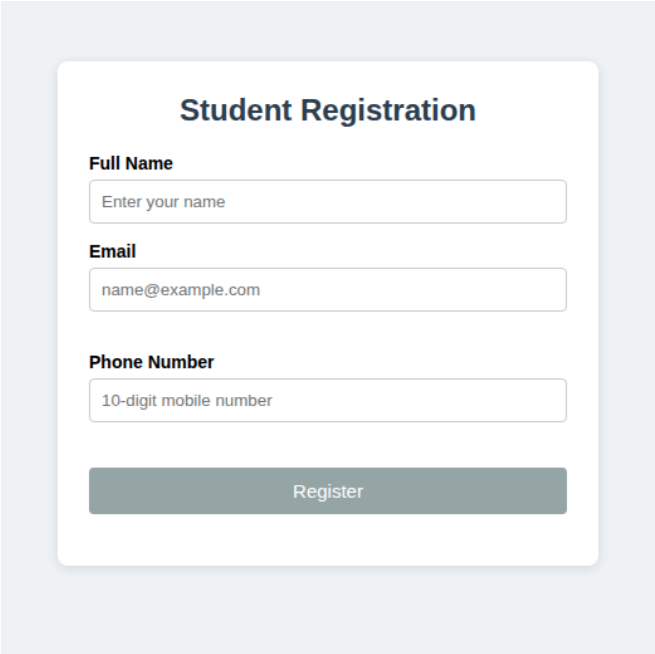


Fig 2.1 – Form on load (Register button greyed out)

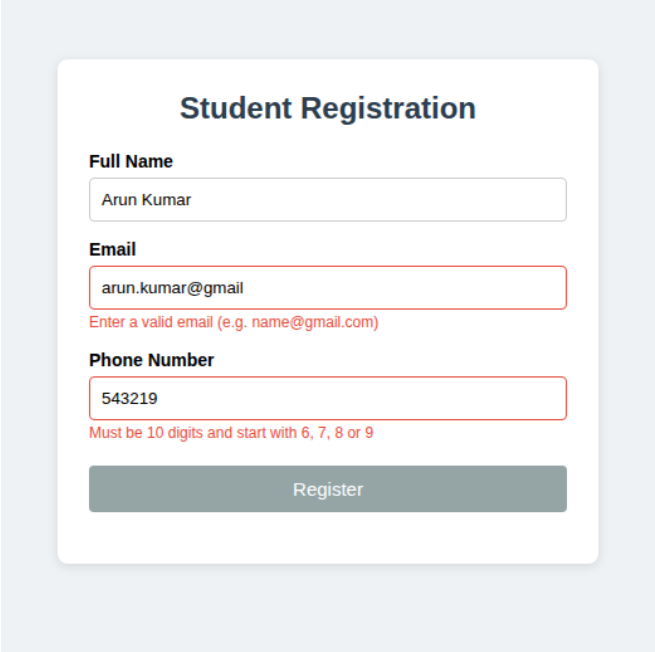


Fig 2.2 – Invalid input: "arun.kumar@gmail" and "54321ab9" (letters removed automatically, both errors shown)

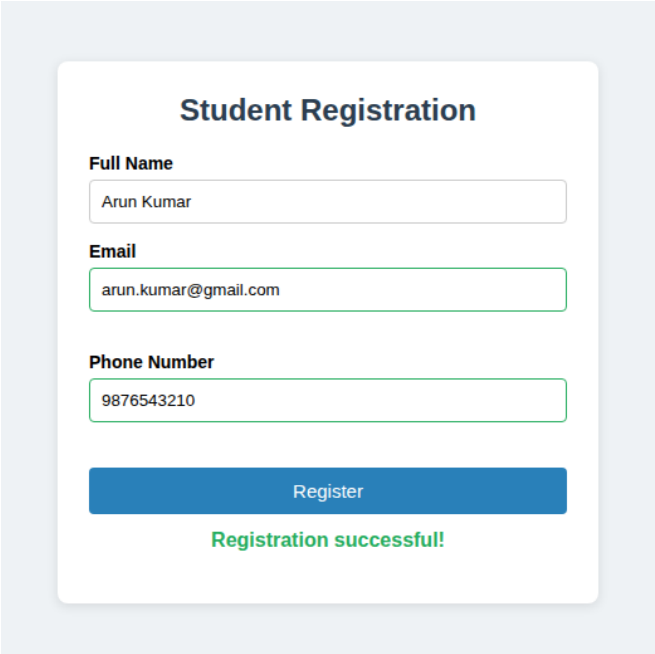


Fig 2.3 – Valid input: "arun.kumar@gmail.com" and "9876543210", then Register clicked

(e) Evaluation and suggested enhancements

I tried the following test inputs:

Input	Field	Expected	Result
`arun.kumar@gmail.com`	Email	Valid	Accepted
`arun.kumar@gmail`	Email	Invalid	Error shown
`arun@.com`	Email	Invalid	Error shown
`9876543210`	Phone	Valid	Accepted
`54321ab9`	Phone	Invalid	Letters stripped, error shown
`1234567890`	Phone	Invalid (starts with 1)	Error shown
(empty)	Both	Invalid	"required" message

All seven cases behaved as expected, and invalid data could not be submitted because the button stays disabled.

Enhancements I would add:

- **Server-side validation** with the same rules, since anyone can turn off JavaScript or send a request directly.
- Show errors on `blur` for the first time instead of on the very first keystroke, so the user isn't shown a red message before they have finished typing.
- Add `aria-live="polite"` on the error spans so screen readers announce the messages.
- Confirm the email with an OTP / verification link – a regex only checks the format, not that the address exists.
- Trim spaces and sanitise inputs before storing them to prevent injection attacks.
- Add a CAPTCHA or rate limit to stop bots from spamming the form.

Experiment 3: CSS Layout Debugging

Aim: To find out why an admin dashboard page overlaps when the browser is resized, and fix it using the box model, Flexbox and Grid.

(a) CSS properties causing the overlap

The dashboard looked almost fine on a wide screen, but when I shrank the window to 600 px the content slid over the sidebar, the footer covered one of the cards, and a horizontal scrollbar appeared.

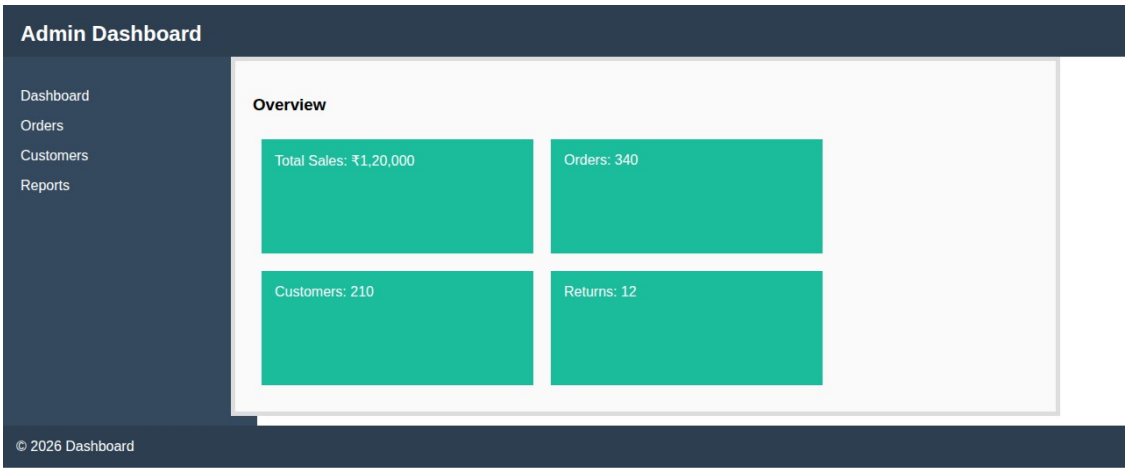


Fig 3.1 – Before, desktop (1280 px): the content box already cuts into the edge of the sidebar

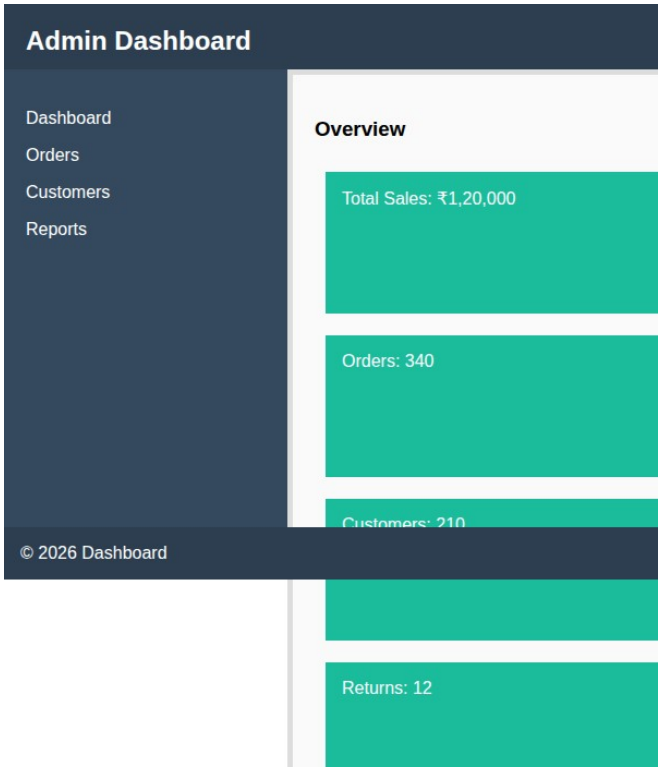


Fig 3.2 – Before, resized (600 px): the footer sits on top of the "Customers" card and the content runs off-screen

The broken CSS was:

```
.header  { position: fixed; top: 0; width: 100%; height: 60px; padding: 0 20px; }
.sidebar { position: absolute; top: 60px; left: 0; width: 250px; height: 400px; padding:
20px; }
.content { position: absolute; top: 60px; left: 260px; width: 70%; padding: 20px; border:
```

```
5px solid #ddd; }
.box     { float: left; width: 280px; height: 100px; margin: 10px; padding: 15px; }
.footer  { position: absolute; top: 480px; width: 100%; padding: 15px; }
```

The properties responsible:

1. **`position: absolute` on sidebar, content and footer.** Absolutely positioned elements are taken out of the normal flow, so they don't push each other – they just stack on top of one another.
2. **Hard-coded `top: 480px` on the footer.** When the content gets taller on a narrow screen, the footer stays at 480 px and covers it.
3. **`left: 260px` on the content** while the sidebar is actually 290 px wide (see box model below), so they overlap by 30 px.
4. **`width: 70%` plus `left: 260px` plus padding and border** add up to more than 100% on small screens, causing horizontal scroll.
5. **`width: 100%` + padding on header/footer** makes them 40 px / 30 px wider than the screen.
6. **Fixed `height: 400px` on the sidebar** and **floats on the cards**, which don't adjust to content size.

(b) Role of the CSS box model

Every element is a box made of four layers: **content → padding → border → margin**. By default (`box-sizing: content-box`) the `width` you write only applies to the content, and padding and border are added on top.

Element	Written width	Padding	Border	Actual width on screen
Sidebar	250 px	20 + 20	0	**290 px**
Card (.box)	280 px	15 + 15	0	310 px (+ 20 px margin = 330 px space)
Content (at 600 px window)	70% = 420 px	20 + 20	5 + 5	**470 px**

This is the real reason the content "started" inside the sidebar – the sidebar was 40 px wider than its CSS said. Setting `box-sizing: border-box` makes padding and border part of the declared width, so the numbers I write are the numbers I get. Margins are still outside the box, and vertical margins between blocks can collapse, which is worth remembering when spacing sections.

(c) Debugging with positioning techniques

I used Chrome DevTools to inspect each element. The box-model diagram in the Computed tab showed the extra padding, and hovering over elements highlighted where they overlapped. Then I decided which positioning to use for each part:

Part	Before	After	Reason
Header	`fixed`	`sticky`	Stays on top while scrolling but still takes up space, so nothing hides under it
Sidebar + content	`absolute`	Flexbox (`display: flex`)	Siblings sit side by side and push each other instead of overlapping
Cards	`float`	CSS Grid `auto-fit`	Wraps to fewer columns automatically as width shrinks
Footer	`absolute; top: 480px`	Normal flow	Always comes after the content, whatever its

Part	Before	After	Reason
			height

`position: relative` is still useful when you need a small offset or a parent for an absolutely positioned badge or icon, but it should not be used to build the main page structure.

(d) Modified CSS (fixed layout)

```
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; display: flex; flex-direction: column;
min-height: 100vh; }

.header  { position: sticky; top: 0; z-index: 10; background: #2c3e50; color: #fff;
padding: 0 20px; }
.layout  { display: flex; flex: 1; }
.sidebar { flex: 0 0 220px; background: #34495e; color: #fff; padding: 20px; }
.content { flex: 1; min-width: 0; padding: 20px; border: 5px solid #ddd; background:
#fafafa; }

.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 16px;
}
.box    { padding: 15px; min-height: 100px; background: #1abc9c; color: #fff; border-
radius: 6px; }
.footer { background: #2c3e50; color: #fff; padding: 15px; text-align: center; }

@media (max-width: 700px) {
  .layout  { flex-direction: column; }
  .sidebar { flex-basis: auto; display: flex; gap: 15px; flex-wrap: wrap; }
  .sidebar p { margin: 0; }
}
```

The HTML was changed slightly: the sidebar and content are now wrapped in a `<div class="layout">` and the four cards in a `<div class="cards">`.



Fig 3.3 – After, desktop (1280 px): sidebar and content side by side, all four cards in one row

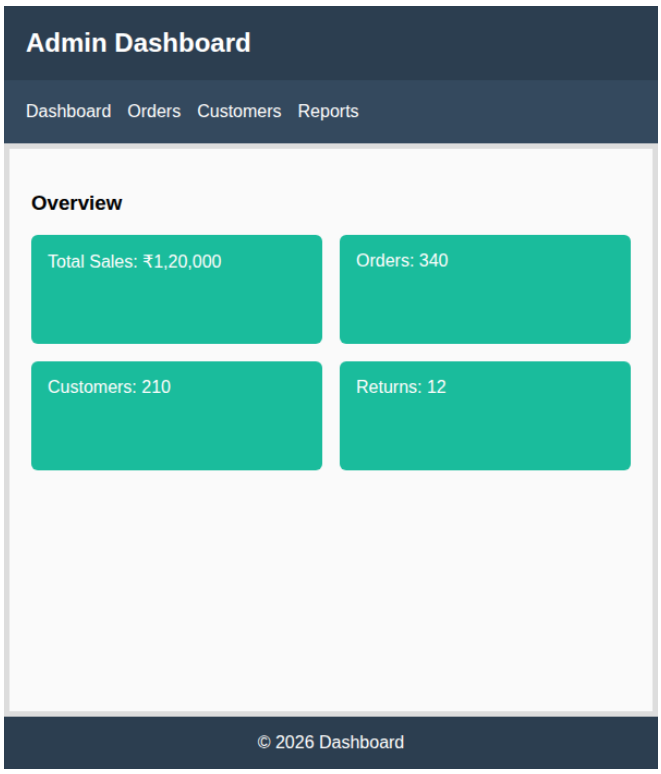


Fig 3.4 – After, resized (600 px): sidebar becomes a horizontal menu, cards in two columns, footer below everything

(e) Evaluation and best practices

After the fix I dragged the window from 1400 px down to 320 px. Nothing overlapped at any width, there was no horizontal scrollbar, and the cards went from 4 → 3 → 2 → 1 columns on their own. The footer always stayed under the content, even with all cards stacked.

Check	Before	After
Content overlaps sidebar	Yes (30 px at all widths)	No
Footer covers content at 600 px	Yes	No
Horizontal scroll at 600 px	Yes	No
Cards adjust to width	No (fixed 280 px floats)	Yes (auto-fit grid)

Best practices I'd follow to keep layouts consistent:

- Put `box-sizing: border-box` on everything at the start of every stylesheet.
- Build page structure with Flexbox and Grid; keep `absolute/fixed` for small overlays like dropdowns, modals and badges.
- Avoid fixed `height` and `top` values on containers; use `min-height` so boxes can grow.
- Use `%`, `fr`, `rem` and `minmax()` instead of fixed pixel widths.
- Add `min-width: 0` to flex children that hold wide content so they can shrink.
- Test at several widths with the DevTools device toolbar while developing, not only at the end.
- Keep spacing in a few fixed values (e.g. 8, 16, 24 px) or CSS variables so all pages look the same.

Conclusion

All three experiments were implemented, run in Chrome and checked at different screen sizes and inputs.

- **Experiment 1:** The retail page was rebuilt mobile-first with CSS Grid areas, Flexbox navigation and two media queries. It now shows 1, 2 or 3 product columns on mobile, tablet and desktop with no horizontal scroll.
- **Experiment 2:** The registration form uses the right input types and attributes, two Regular Expressions, and live error messages. All 7 test inputs behaved as expected, and invalid data cannot be submitted.
- **Experiment 3:** The overlap came from absolute positioning, a fixed footer position and the default content-box sizing. Switching to `border-box`, Flexbox and an `auto-fit` grid removed every overlap from 1400 px down to 320 px.

The main thing I learned is that layout problems usually come from fixed sizes and taking elements out of the normal flow. Letting the browser size things with Flexbox and Grid, and checking the box model in DevTools, solves most of them.